Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
2	**Part A — Practice Programs (Hour 1)** Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. **1. Smart thermostat (Thermostat.java, main in this file)** (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)". (e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)". (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). **Expected output:** *temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.* **2. Cinema show (CinemaShow.java, main in this file)** (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked(). **Expected output:** *booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.* **3. (For Advanced Learners – home) Parking lot (ParkingLot.java)** (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to revenue; for "four", if fourWheelers < fourCap increment and add 40; otherwise print "Full" and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue. **Expected output:** *parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).* **Part B — Project Work (Hour 2): MiniBank** In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can change only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends. 1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods.	**CO1** PO1, PO2, PO3

Practical Number		CO/PO
	2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001"). 3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance. 5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance. **Key Questions / Analysis / Interpretation to be evaluated during/after Implementation** 1. What is encapsulation, and how do private fields with public getters enforce it? 2. Why is accountNumber declared final? 3. What is the difference between a static field and an instance field when generating ids? **Supplementary Problems –** • Add a helper that moves an amount from one Account to another using withdraw and deposit. • Reject a deposit or withdrawal of a negative amount (return without changing the balance). **Key Skills to be addressed –** Defining classes and objects; instance vs static members; constructors and constructor chaining with this(); access modifiers; encapsulation; arrays of objects. **Applications –** Modelling real-world entities (accounts, customers, products) with controlled, valid state is the foundation of every business application. **Learning Outcome –** The student can design well-encapsulated classes and create and manipulate objects safely. (CO1)	
	Dataset / Test Data (Source and Description, if applicable): No external dataset. Sample objects are created in code (for example owner "Riya" with opening balance 5000).	
	Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git.	
	• **Total Hours of Problem Definition Implementation:** 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • **Total Hours of Engagement** = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-02. Confirm that balance cannot be modified from outside the class and that withdraw returns false when funds are insufficient.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct classes and encapsulation – 40%; constructors and static id generation – 25%; deposit/withdraw logic – 20%; naming conventions and viva – 15%.	
	Expected Input / Process / Output **Input:** In code: create an account with opening balance 5000, then call deposit(2000), withdraw(3000) and withdraw(10000). **Process:** deposit adds to the balance; the first withdraw succeeds because the balance is sufficient and reduces it; the second withdraw fails and leaves the balance unchanged. **Expected Output:** Balance prints as 7000 after the deposit, 4000 after the successful withdraw, and remains 4000 after the failed withdraw (which returns false).	